- **Internship Name : Data Analysis**
- **Task Name: Task 12: Data Visualization using Matplotlib**
- **Your Name : Shubhangi Naresh Khonde**
- **Internship Organization :**
- **Date:**

1. Title

Creating Line, Bar, and Scatter Plots using Matplotlib on a Sample Dataset

2. Aim

To create line, bar, and scatter plots using the Matplotlib library in Python, and to format them with appropriate titles, axis labels, and legends.

3. Objectives

- To understand the basics of data visualization using Matplotlib.
- To plot a line chart, a grouped bar chart, and a scatter plot.
- To apply formatting such as titles, labels, legends, grid lines, colors, and markers.
- To interpret the information shown by each type of plot.

4. Introduction

Data visualization is the graphical representation of data. It helps in identifying trends, comparing values, and understanding relationships between variables. Matplotlib is a widely used Python library for creating static, animated, and interactive visualizations. Its pyplot module provides simple functions to build and customize different types of charts.

- **Line plot:** shows how a value changes over time or an ordered sequence.
- **Bar plot:** compares values across different categories.
- **Scatter plot:** shows the relationship between two numerical variables.

5. Tools and Technologies Used

- Programming language: Python 3
- Libraries: Matplotlib, NumPy, Pandas
- Environment: Jupyter Notebook / any Python IDE

6. Dataset Description

Two sample datasets were created for this task.

Dataset 1: Monthly sales of two products (units)

```
##Step 2: Created Sample dataset

    Months = ["Jan","Feb","Mar","Apr","May","Jun"]
    Sales_a = [120,150,170,160,200,230]  #Product A
    Sales_b = [100,130,120,180,170,210] #Product B

    #Scatter : Study Hours vs Marks
    Hours = np.array([1,2,3,4,5,6,7,8,9,10])
    Marks = np.array([35,40,50,55,60,68,72,80,85,92])
```

Month	Product A	Product B
Jan	120	100
Feb	150	130
Mar	170	120
Apr	160	180
May	200	170
Jun	230	210

Dataset 2: Study hours vs marks obtained

Study Hours	1	2	3	4	5	6	7	8	9	10
Marks	35	40	50	55	60	68	72	80	85	92

7. Methodology / Procedure

1. Import the required libraries: matplotlib.pyplot, numpy, and pandas.
2. Create the sample datasets as lists and NumPy arrays.
3. Create a line plot using plt.plot() with markers and different line styles for each product.
4. Create a grouped bar plot using plt.bar() by shifting the bar positions for the two products.
5. Create a scatter plot using plt.scatter() and add a trend line using np.polyfit().
6. Format each plot using plt.title(), plt.xlabel(), plt.ylabel(), plt.legend(), and plt.grid().
7. Display the plots using plt.show() and save the figure using plt.savefig().

8. Formatting Techniques Used

Feature	Function Used	Purpose
Title	plt.title()	Describes what the chart represents
Axis labels	plt.xlabel(), plt.ylabel()	Identify the variables on each axis
Legend	plt.legend()	Distinguishes multiple data series
Grid	plt.grid()	Improves readability of values
Markers and colors	marker=, color=	Highlight data points and separate series
Figure size	plt.figure(figsize=())	Controls the size of the output

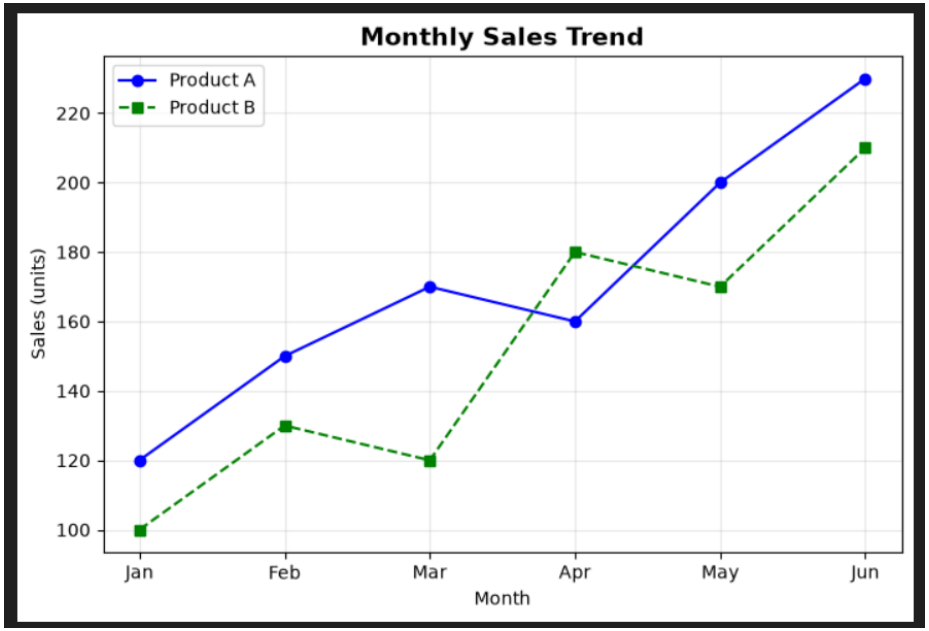
8. Output:

- **Figure 1:** Line plot showing the monthly sales trend of Product A and Product B.

```
##Step 3: Line Plot

    plt.figure(figsize=(8, 5))
    plt.plot(Months, Sales_a, marker="o", color="blue", label="Product A")
    plt.plot(Months, Sales_b, marker="s", color="green", linestyle="--", label="Product B")

    plt.title("Monthly Sales Trend", fontsize=14, fontweight="bold")
    plt.xlabel("Month")
    plt.ylabel("Sales (units)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()
```



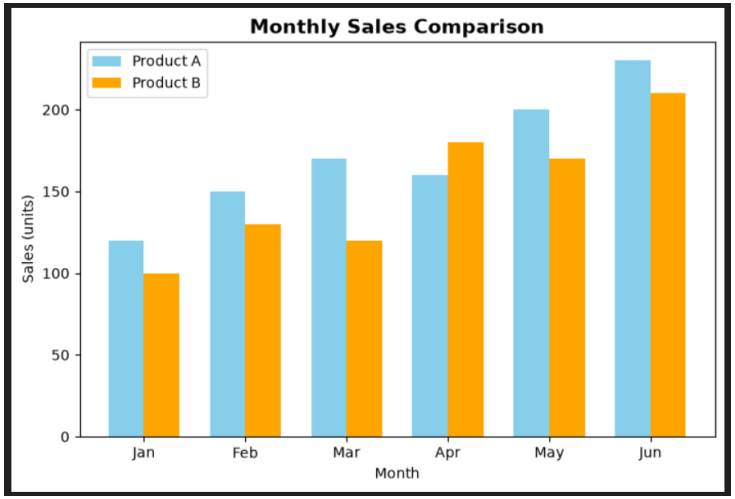
- **Figure 2:** Grouped bar plot comparing monthly sales of both products.

```
##Step 4: Bar Plot (Grouped)

    x = np.arange(len(Months))   # [0,1,2,3,4,5]
    width = 0.35                 # bar chi runda

    plt.figure(figsize=(8, 5))
    plt.bar(x - width/2, Sales_a, width, label="Product A", color="skyblue")
    plt.bar(x + width/2, Sales_b, width, label="Product B", color="orange")

    plt.xticks(x, Months)        # numbers aivaji month names
    plt.title("Monthly Sales Comparison", fontsize=14, fontweight="bold")
    plt.xlabel("Month")
    plt.ylabel("Sales (units)")
    plt.legend()
    plt.show()
```



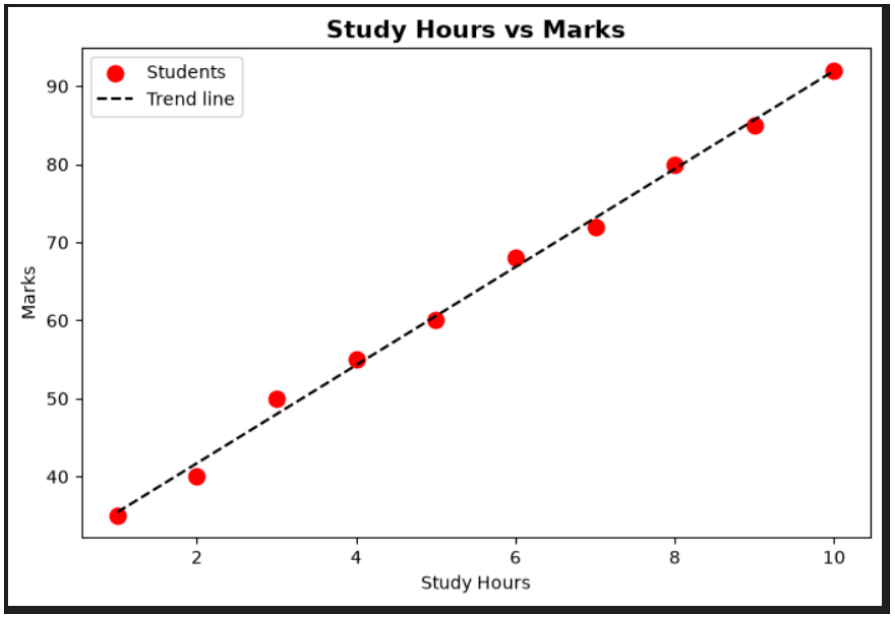
- **Figure 3:** Scatter plot of study hours versus marks, with a trend line.

```
##Step 5: Scatter Plot

    plt.figure(figsize=(8, 5))
    plt.scatter(Hours, Marks, color="red", s=80, label="Students")

    # Optional: trend line
    m, c = np.polyfit(Hours, Marks, 1)
    plt.plot(Hours, m*Hours + c, color="black", linestyle="--", label="Trend line")

    plt.title("Study Hours vs Marks", fontsize=14, fontweight="bold")
    plt.xlabel("Study Hours")
    plt.ylabel("Marks")
    plt.legend()
    plt.show()
```



9. Observations

- **Line plot:** Sales of both products increased from January to June. Product A showed steady growth, while Product B fluctuated, with a dip in March and a sharp rise in April.
- **Bar plot:** Product A had higher sales than Product B in most months. Product B exceeded Product A only in April.
- **Scatter plot:** The points rise from the lower left to the upper right, showing a strong positive correlation between study hours and marks. The trend line confirms that marks increase as study hours increase.

10. Conclusion

Line, bar, and scatter plots were successfully created using Matplotlib. Each plot type suits a different purpose: line plots for trends, bar plots for comparisons, and scatter plots for relationships between variables. Adding titles, axis labels, and legends makes the charts clear and easy to interpret. Matplotlib proved to be a simple and flexible tool for data visualization in Python.